



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1-4

по дисциплине «Системная и программная инженерия»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Руководитель проектной работы

Мельников Д.А.

(подпись руководителя)

Работа представлена

«___» _____ 2026 г.

Допущен к работе

«___» _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	2
1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА.....	4
1.1 Техническое задание	4
1.1.1 Общие сведения	4
1.1.2 Цели и назначение автоматизированной системы	4
1.1.3 Характеристика объектов автоматизации	4
1.1.5 Состав и содержание работ	11
1.1.6 Порядок разработки и приемки	12
1.1.7 Требования к подготовке объекта к вводу системы в действие	12
1.1.8 Требования к документированию	12
1.1.9 Источники разработки.....	12
1.2 Целевая аудитория	12
1.2.1 Основной портрет	12
1.2.2 Второстепенный портрет.....	13
1.3 Диаграмма вариантов использования	13
1.3.1 Обоснование сценариев использования	13
1.3.2 Схематическое представление (Use Case Diagram)	13
1.4 Диаграмма последовательности	14
1.5 План реализации проекта	15

ВВЕДЕНИЕ

Современная жизнь требует от человека, особенно от студента, грамотного управления личными финансами. Проект "Finlytics: Офлайн-менеджер финансов" на языке Kotlin направлен на разработку десктопного приложения, которое предоставит пользователю удобный инструмент для учета доходов и расходов, категоризации трат и визуализации финансовой статистики без необходимости подключения к интернету. Основной целью является создание простого, интуитивно понятного и производительного приложения, которое помогает пользователям контролировать свой бюджет.

Краткое описание проекта:

Проект представляет собой десктопное приложение "Finlytics" для управления персональными финансами. Ключевые функции: внесение операций "доход" и "расход" с возможностью их категоризации, фильтрация данных по различным временным промежуткам, а также наглядное представление финансовой статистики в виде диаграмм и списков.

Проблематика:

Актуальность проекта обусловлена необходимостью простого и доступного инструмента для планирования и контроля личного бюджета. Многие существующие решения либо требуют постоянного подключения к интернету, либо обладают избыточной сложностью, либо неудобны в повседневном использовании. Студенты, как социальная группа с часто ограниченным бюджетом, особенно нуждаются в таком инструменте для формирования навыков финансовой грамотности.

Для достижения цели необходимо выполнить следующие задачи:

- Задача 1: Разработать архитектуру данных и пользовательский интерфейс для удобного внесения и редактирования финансовых операций.

- Задача 2: реализовать механизм категоризации доходов и расходов с возможностью добавления пользовательских категорий.
- Задача 3: внедрить систему фильтрации и анализа данных по временным промежуткам (день, неделя, месяц, год, все время) с визуализацией результатов в виде диаграмм и отчетов.

Состав команды и распределение ролей:

- Враженко Д.О. – Team Lead, менеджер проекта, разработчик, тестировщик, технический писатель. Ответственный за постановку задач, общую архитектуру проекта, разработку ключевых модулей и составление отчетности.
- Хохряков А.Ю. – Аналитик, разработчик, тестировщик, технический писатель. Ответственный за реализацию модуля работы с данными и бизнес-логики.
- Павлов Н.С. – UI/UX дизайнер, разработчик, тестировщик, технический писатель. Ответственный за проектирование и реализацию пользовательского интерфейса, включая визуализацию данных (диаграммы).

1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА

1.1 Техническое задание

1.1.1 Общие сведения

Наименование проекта: "Finlytics".

Team Lead: Враженко Д.О.

Разработчики: Враженко Д.О., Павлов Н.С., Хохряков А.Ю.

Заказчик: Враженко Д.О.

Язык разработки: Kotlin.

База данных: SQLite

Целевая платформа: Десктоп.

Тип приложения: Офлайн-приложение.

1.1.2 Цели и назначение автоматизированной системы

Цель: Создать удобный и наглядный инструмент "Finlytics" для ведения учета личных финансов, не требующий подключения к интернету.

Назначение: Автоматизация процесса учета доходов и расходов, предоставление пользователю аналитики о своей финансовой деятельности для принятия обоснованных решений. Система предназначена для широкого круга пользователей, но особенно актуальна для студентов.

1.1.3 Характеристика объектов автоматизации

Объектом автоматизации является процесс персонального финансового учета, который включает в себя:

- Регистрацию финансовых операций (доходы и расходы).
- Категоризацию операций.
- Фильтрацию и поиск операций по дате и категориям.
- Анализ и визуализацию финансовых данных.

1.1.4 Требования к системе

1.1.4.1 User Story

Таблица 1 – User Story

Кто?	Что хочет?	С какой целью?
Пользователь	Чтобы при первом запуске в приложении уже был предустановленный набор категорий	Чтобы не приходилось тратить время на их создание с нуля, и пользователь мог сразу начать вносить свои траты
Пользователь	Быстро добавить информацию о том, что потратил деньги, указав сумму, категорию и дату	Чтобы точно фиксировать все траты
Пользователь	Внести запись о получении денег	Чтобы видеть полную картину своих финансов
Пользователь	Иметь возможность создавать свои собственные категории для доходов и расходов	Чтобы не ограничиваться предустановленными категориями
Пользователь	Когда открывается главный экран, отображать, какой баланс, а также общую сумму доходов и расходов за выбранный период	Чтобы быстро оценить свое финансовое состояние
Пользователь	Видеть наглядную диаграмму, которая показывает, на что именно тратятся деньги больше всего	Чтобы понимать структуру своих расходов
Пользователь	Иметь возможность посмотреть свою финансовую статистику и историю операций не только за всё время, но и за конкретный период (день, неделю, месяц или год)	Чтобы отслеживать динамику трат и доходов
Пользователь	Иметь возможность открыть запись и редактировать её	Чтобы устранить ошибку при записи данных
Пользователь	Иметь возможность удалить запись из истории	Чтобы она не учитывалась в общей статистике
Пользователь	Иметь возможность удалять созданные мной категории	Чтобы убрать неиспользуемые категории
Пользователь	Иметь отдельный экран, где отображается полный список всех моих финансовых операций в хронологическом порядке, с возможностью их фильтрации	Чтобы отследить свои операции за конкретный период

1.1.4.2 Функциональные требования

На основе описанных User Story были сформулированы следующие функциональные требования проекта:

- Система должна автоматически создавать предустановленный список категорий доходов и расходов при первом запуске приложения (инициализация БД).
- Пользователь должен иметь возможность создавать новые категории доходов и расходов.
- Пользователь должен иметь возможность удалять существующие категории.
- Система должна блокировать удаление категории, если она используется хотя бы в одной операции.
- Пользователь должен иметь возможность добавлять операции типа "Доход" и "Расход".
- Операция должна содержать: сумму, категорию, дату и описание (необязательно).
- Система должна валидировать вводимые данные: сумма должна быть положительным числом, дата — корректной, категория — выбрана из списка.
- Пользователь должен иметь возможность редактировать любую существующую операцию.
- Пользователь должен иметь возможность удалять любую существующую операцию.
- На главном экране отображать сводку: баланс, общие доходы и расходы за выбранный период.
- На главном экране отображать круговую диаграмму распределения операций по категориям (с переключением "Доходы/Расходы").
- На экране истории отображать полный список операций.

- Реализовать фильтрацию данных на главном экране и в истории по временным промежуткам (день, неделя, месяц, год, все время).
- Реализовать фильтрацию в истории по типу операции (Все/Доходы/Расходы).

1.1.4.3 Нефункциональные требования

Учитывая, что проект представляет собой десктопное офлайн-приложение, были определены следующие нефункциональные требования:

Таблица 2 – Нефункциональные требования

Тип требования	Содержание требования
Технические ограничения	Десктопное приложение должно функционировать на операционных системах семейства Windows (10 и выше), macOS (11 и выше) и Linux (дистрибутивы с поддержкой JRE).
	Приложение должно распространяться в виде исполняемого JAR-файла, для работы которого требуется Java Runtime Environment (JRE) версии 8 или выше.
	Для хранения данных должно использоваться исключительно локальное хранилище (SQLite) без необходимости подключения к интернету.
Производительность	Время отклика интерфейса на действия пользователя (открытие экрана, добавление операции, применение фильтра) не должно превышать 500 миллисекунд.
	Запуск приложения и загрузка начальных данных (история операций и категории) должны занимать не более 3 секунд.
	Операции фильтрации списка транзакций и пересчета статистики по временным периодам должны выполняться мгновенно, без заметных задержек.
Надежность	Приложение должно корректно завершать работу и сохранять целостность данных при закрытии окна или выключении компьютера.
	Сбои в работе отдельных компонентов (например, при попытке удалить используемую категорию) не должны приводить к полной остановке или некорректному состоянию приложения.
Безопасность	Локальная база данных (SQLite) не должна быть доступна для чтения или изменения другими приложениями напрямую (защита на уровне файловой системы).
	Приложение должно блокировать SQL-инъекции при формировании запросов к базе данных (за счет использования параметризованных запросов в коде).
Удобство использования	Интерфейс приложения должен быть интуитивно понятным и не требовать от пользователя изучения дополнительной документации для выполнения основных сценариев (добавление, редактирование, удаление).

	Все ключевые элементы управления (кнопки, поля ввода) должны быть снабжены всплывающими подсказками (ToolTips), поясняющими их назначение.
	Система должна предоставлять пользователю понятные сообщения об ошибках ввода данных (например, при вводе некорректной суммы или даты).

1.1.4.4 Матрица требований

Матрица объединяет функциональные требования, выявленные на этапе планирования, и нефункциональные требования, сформулированные выше. Она структурирована по основным компонентам приложения и источникам.

Таблица 3 – Матрица требований

№	Требование	Суть	Автор	Ссылки	Критерии проверки
1	Клиентское приложение (Desktop)				
1.1	Управление операциями				
1.1.1	Добавление операции	Пользователь должен иметь возможность добавить новую финансовую операцию с указанием типа (доход/расход), суммы, категории, даты и описания.	Павлов Н.С.	User Story (Таблица 1 отчета)	При нажатии на кнопку "Добавить" открывается диалоговое окно, в котором можно ввести все параметры и сохранить операцию.
1.1.2	Редактирование операции	Пользователь должен иметь возможность открыть существующую запись и изменить любые её параметры.	Павлов Н.С.	User Story (Таблица 1 отчета)	При нажатии на кнопку "Редактировать" у элемента списка открывается диалог с предзаполненными полями, после изменения и сохранения данные обновляются.
1.1.3	Удаление операции	Пользователь должен иметь возможность удалить запись из истории, при этом она	Павлов Н.С.	User Story (Таблица 1 отчета)	При нажатии на кнопку "Удалить" появляется диалог подтверждения.

		перестает учитываться в статистике.			После подтверждения операция исчезает из списка и не влияет на баланс и диаграммы.
1.2	Управление категориями				
1.2.1	Предустановленные категории	При первом запуске в приложении должен быть предустановленный набор категорий для доходов и расходов.	Хохряков А.Ю.	User Story (Таблица отчета) 1	После чистой установки и первого запуска в настройках отображаются категории: Зарплата, Стипендия, Еда, Транспорт и др.
1.2.2	Создание пользовательских категорий	Пользователь должен иметь возможность создавать свои собственные категории для доходов и расходов.	Хохряков А.Ю.	User Story (Таблица отчета) 1	В экране "Настройки" при нажатии на кнопку "+" появляется диалог, позволяющий ввести имя новой категории и добавить её в соответствующий список.
1.2.3	Удаление категорий	Пользователь должен иметь возможность удалять созданные им категории. Система должна блокировать удаление категории, если она используется в операциях.	Хохряков А.Ю.	User Story (Таблица отчета) 1	При попытке удалить категорию, которая привязана к какой-либо операции, должно появляться предупреждение, и удаление не должно выполняться.
1.3	Аналитика и просмотр				
1.3.1	Финансовая сводка	На главном экране должна отображаться сводка: баланс, общая сумма доходов и расходов за выбранный	Враженко Д.О.	User Story (Таблица отчета) 1	На экране "Обзор" при выборе периода (День/Неделя/Месяц) цифры в блоках "Доходы", "Расходы" и

		период.			"Баланс" пересчитываются .
1.3.2	Визуализация данных	На главном экране должна отображаться круговая диаграмма, показывающая структуру доходов или расходов по категориям за выбранный период.	Вражен ко Д.О.	User Story 1 (Таблица отчета)	При переключении между режимами "Доходы" и "Расходы" на главном экране меняется диаграмма и список категорий, отображая соответствующие данные.
1.3.3	Фильтрация истории	На экране истории пользователь должен иметь возможность фильтровать список операций по временному промежутку (день, неделя, месяц, год, все время) и по типу операции (все, доходы, расходы).	Вражен ко Д.О.	User Story 1 (Таблица отчета)	При изменении параметров фильтрации на экране "История" список операций должен обновляться в соответствии с выбранными критериями.
2	Правовые нормы и стандарты				
2.1	Защита персональных данных	Локальная база данных с персональными финансовыми данными пользователя не должна быть доступна для чтения сторонними приложениями.	Вражен ко Д.О.	ГОСТ 34.602-2020, Федеральный закон № 152-ФЗ "О персональных данных"	При проверке файловой системы, файл базы данных (Finlytics.db) не должен открываться стандартными средствами ОС или другими программами без специального доступа.
2.2	Соответствие стандартам документирования	Исходный код должен быть документирован в соответствии со стандартом	Вражен ко Д.О.	ГОСТ 34.201-2020, официальная документация Kotlin	При проверке исходного кода, все публичные классы и функции должны содержать KDoc-

		KDoc для языка Kotlin.			комментарии с описанием.
3	Технические и нефункциональные ограничения				
3.1	Кроссплатформенность	Приложение должно работать в ОС Windows, macOS и Linux.	Враченко Д.О.	Анализ рынка аналоговых приложений	Собранный JAR-файл успешно запускается и корректно отображает интерфейс, а также выполняет все функции на каждой из трех целевых ОС.
3.2	Производительность фильтрации	Операции фильтрации данных по временным промежуткам должны выполняться мгновенно, без заметных задержек для пользователя.	Павлов Н.С.	Нефункциональные требования (п. 1.1.4.1)	При переключении между вкладками "День/Неделя/Месяц" на экране "Обзор" или "История" интерфейс обновляется быстрее, чем за 0.5 секунды.
3.3	Безопасность данных	Приложение должно использовать параметризованные SQL-запросы для исключения возможности повреждения БД некорректным вводом.	Хохряков А.Ю.	Нефункциональные требования (п. 1.1.4.1)	Проверка кода: все обращения к БД в объектах AddCategory, AddIncomeTransaction и др. должны использовать PreparedStatement с плейсхолдерами (?).

1.1.5 Состав и содержание работ

Проектирование пользовательского интерфейса (UI/UX).

Разработка объектной модели данных.

Реализация модуля для хранения данных.

Реализация бизнес-логики (добавление, удаление, фильтрация операций, расчет статистики).

Создание модуля визуализации данных (диаграммы).

Интеграция всех модулей, тестирование и отладка.

1.1.6 Порядок разработки и приемки

Разработка ведется итерационно. Каждая итерация включает в себя реализацию одного из пунктов состава работ, его тестирование и интеграцию. Приемка этапов осуществляется лидером проекта (Враженко Д.О.).

1.1.7 Требования к подготовке объекта к вводу системы в действие

Приложение будет распространяться в виде исполняемого JAR-файла, для работы которого требуется JRE версии 8 или выше. В качестве альтернативы рассматривается создание нативного исполняемого файла с помощью Kotlin/Native для упрощения развертывания.

1.1.8 Требования к документированию

Исходный код должен быть документирован в соответствии с общепринятыми стандартами для Kotlin.

Должна быть предоставлена краткая инструкция пользователя внутри приложения (например, в виде ToolTip'ов).

1.1.9 Источники разработки

Официальная документация по языку Kotlin.

Анализ существующих аналоговых приложений.

1.2 Целевая аудитория

1.2.1 Основной портрет

Студент, начинающий контролировать личный бюджет.

Это основная целевая аудитория. Данный пользователь — молодой человек или девушка, часто впервые сталкивающийся с необходимостью самостоятельного управления финансами (стипендия, подработки, деньги от родителей). Его бюджет ограничен, и он ищет простой и понятный

инструмент, чтобы отслеживать, на что уходят деньги, и учиться планировать траты.

1.2.2 Второстепенный портрет

Занятый профессионал, ценящий приватность и офлайн-доступ.

Этот пользователь — работающий человек, который уже имеет некоторый финансовый опыт, но предпочитает держать свои данные под полным контролем. Он ценит скорость и эффективность, у него нет времени на изучение сложного софта. Ему нужен надежный инструмент для быстрой фиксации основных операций и получения сводки без подключения к интернету.

1.3 Диаграмма вариантов использования

1.3.1 Обоснование сценариев использования

Пользователь добавляет новую операцию (доход/расход).

Пользователь редактирует существующую операцию.

Пользователь удаляет операцию.

Пользователь просматривает историю операций.

Пользователь фильтрует операции по дате и категории.

Пользователь просматривает финансовую сводку.

Пользователь управляет категориями.

1.3.2 Схематическое представление (Use Case Diagram)

Диаграмма включает одного основного актора – Пользователь. Сценарии использования: "Добавить операцию", "Редактировать операцию", "Удалить операцию", "Просмотреть историю операций", "Отфильтровать операции", "Просмотреть финансовую сводку", "Добавить категорию".

Результат создания диаграммы прецедентов представлен на Рисунке 1.



Рисунок 1 — Диаграмма прецедентов разрабатываемого проекта

1.4 Диаграмма последовательности

Диаграмма включает 4 объекта – Пользователь, Экран приложения (UI), Менеджер состояния (ViewModel), Хранилище данных (Repository), База данных SQLite.

Результат создания диаграммы прецедентов представлен на Рисунке 2.

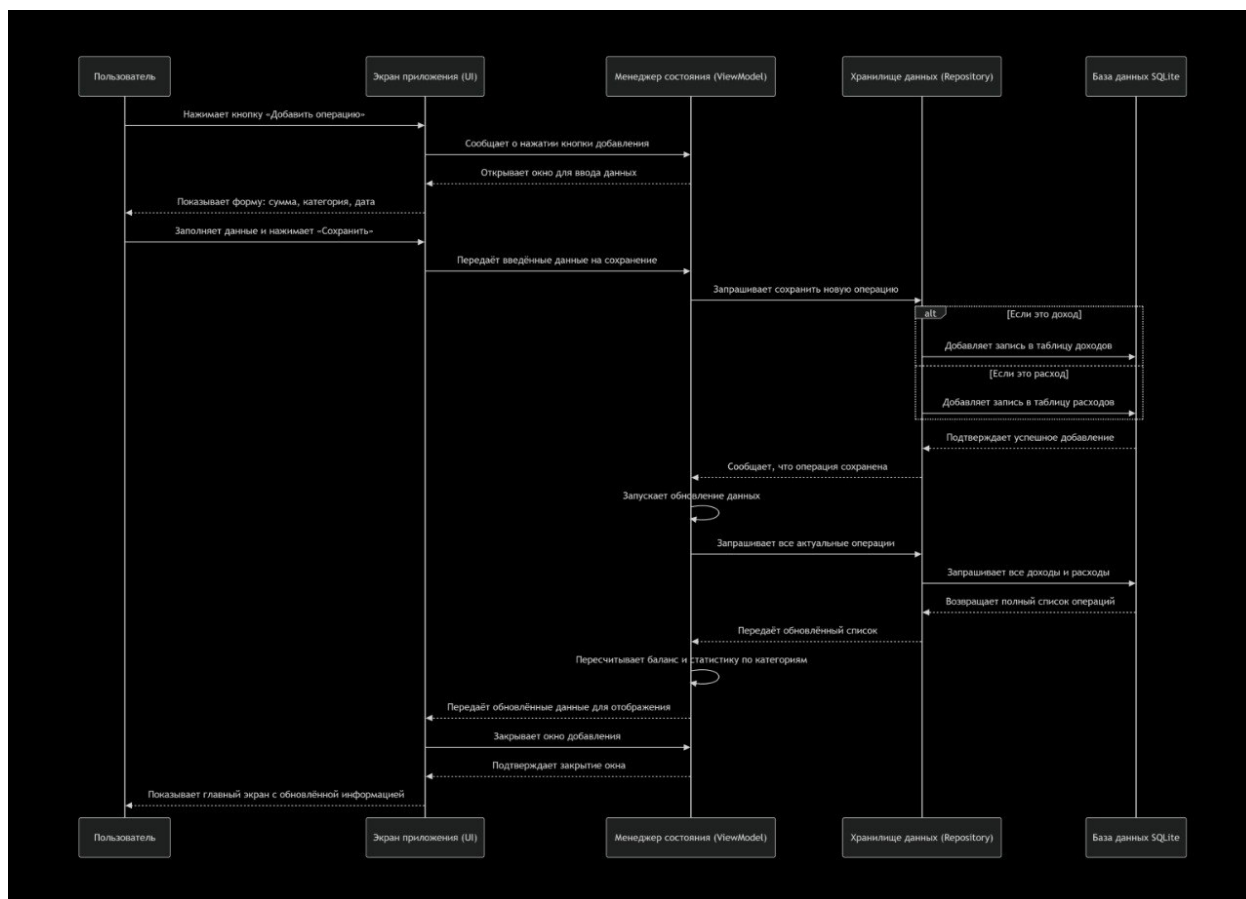


Рисунок 2 — Диаграмма последовательности разрабатываемого проекта

1.5 План реализации проекта

После детального анализа и выявления требований к системе был составлен план работ и распределены задачи и обязанности между участниками проекта в сервисе Miro (Таблица 4).

Таблица 4 – Распределение обязанностей между участниками проекта

Этап	Враженко Д.О.	Хохряков А.Ю.	Павлов Н.С.
Подготовка и проектирование	– Детализация технического задания, разбивка на конкретные задачи – Определение общей архитектуры приложения (MVVM) – Настройка системы отслеживания задач – Конфигурация сборочного файла (Gradle) с необходимыми зависимостями	– Создание и настройка репозитория на GitHub – Проектирование структуры базы данных (таблицы, связи) – Определение моделей данных для финансовых операций и категорий	– Создание макетов всех экранов в Figma – Разработка дизайн-системы (цветовая палитра, типографика, отступы) – Подготовка кликабельного прототипа для демонстрации пользовательского пути

Разработка backend	– Координация разработки, проведение код-ревью – Написание базовой логики ViewModel (управление состоянием, обработка действий пользователя) – Интеграция слоя данных с ViewModel	– Разработка модулей для подключения к SQLite и инициализации базы данных – Создание набора стандартных категорий (доходы/расходы) – Реализация операций CRUD для категорий и транзакций – Формирование единого репозитория для доступа к данным	– Создание базовых UI-компонентов (навигационная панель, карточки, кнопки) – Разработка заготовки для кастомной диаграммы
Разработка пользовательского интерфейса	– Подключение интерфейса к ViewModel (реактивное обновление данных) – Реализация навигации между экранами – Интеграция диалоговых окон	– Поддержка и консультации по работе с данными – Доработка репозитория при необходимости	– Вёрстка главного экрана (обзорная статистика, фильтры, диаграмма) – Вёрстка экрана истории операций (список, фильтрация) – Вёрстка экрана настроек (управление категориями) – Реализация диалогов для добавления/редактирования операций и категорий
4. Интеграция, тестирование и отладка	– Организация процесса тестирования всех сценариев использования – Обработка ошибок и валидация пользовательского ввода – Документирование ключевых компонентов кода	– Тестирование корректности работы с базой данных (целостность, запросы) – Проверка всех операций с категориями и транзакциями – Оптимизация запросов при необходимости	– Тестирование пользовательского интерфейса на соответствие макетам
Финальная сборка и подготовка к сдаче	– Координация финальной сборки – Подготовка отчётной документации по проекту – Презентация готового продукта	– Проверка работоспособности приложения на целевых платформах – Участие в подготовке отчёта	– Участие в подготовке отчёта – Демонстрация дизайн-решений и реализованного интерфейса